Lecture 7 — Advanced Math: Matrix Exponentiation, FFT/NTT, Gaussian Elimination & XOR Bases

> National Championship Prep · Prerequisites: Year weeks 10–11 (number theory, modular inverses, fast power), Year week 30 (combinatorics), Regional 07 (ext. Euclid, CRT) · Estimated time: 2 × 75 min lecture + 3 h practice

1. Overview

Three pieces of linear algebra appear over and over in national-championship sets, usually disguised:

1. **Matrix exponentiation.** Anything of the form "apply the same linear step n times" with n up to 10^{18}. The step can be over integers mod p, or over a *semiring* such as (min, +).
2. **Fast Fourier Transform / Number Theoretic Transform.** Multiplying two polynomials of size 10^5–10^6 in O(N log N). The art is not the FFT (it goes in the notebook) but recognizing that the problem *is* a convolution.
3. **Gaussian elimination**, over the reals, mod a prime, or over GF(2). Its most common contest incarnation is the **XOR basis** (a.k.a. linear basis), which answers max-xor / k-th-xor / "is x a subset xor" questions in O(log C).

For each tool we state why it is correct, what it costs, and the patterns that reveal it in a statement. The World Finals track (Lecture 2) builds on this with polynomial inverse/log/exp, Berlekamp–Massey, and Lagrange interpolation.

2. Matrix exponentiation

2.1 The principle

If a state vector evolves by $v_{t+1} = M v_t$, then $v_n = M^n v_0$, and M^n takes O(log n) matrix multiplications by repeated squaring. With a k×k matrix the cost is $O(k^3 \log n)$. The only property used is **associativity** of matrix multiplication, which holds whenever the entries come from a semiring: an "addition" ⊕ that is associative and commutative, a "multiplication" ⊗ that is associative and distributes over ⊕. Hence:

- (+, ×) mod p: $(A^n)[u][v]$ = number of walks of length exactly n from u to v (CSES 1723 *Graph Paths I*).
- (min, +): $(A^n)[u][v]$ = minimum weight of a walk with exactly n edges (CSES 1724 *Graph Paths II*). The identity matrix has 0 on the diagonal and $+\infty$ elsewhere.
- (OR, AND): reachability in exactly n steps; store rows as `bitset` for a 64× speedup (CF 576D *Flights for Regular Customers* combines this with sorting edges by unlock time).

2.2 Linear recurrences

For $a_i = c_1 a_{i-1} + \ldots + c_k a_{i-k}$ keep the vector $(a_{i-1}, \ldots, a_{i-k})$. The **companion matrix** has the coefficients in its first row and a shifted identity below. Then $(a_{n}, \ldots, a_{n-k+1}) = T^{n-k+1} (a_{k-1}, \ldots, a_0)$.

Non-homogeneous terms become extra coordinates:

- constant $+d$: add a coordinate that is always 1, with d in the first row;
- polynomial $+ i^2$: add coordinates $i^2, i, 1$, using $(i+1)^2 = i^2 + 2i + 1$;
- prefix sums: add $S_i = S_{i-1} + a_i$ as a coordinate.

Exponent recurrences. CF 1182E *Product Oriented Recurrence* defines $f_x = c^{2x-6} f_{x-1} f_{x-2} f_{x-3}$ mod 10^9+7. Taking discrete "logs" informally, the exponents of f_1, f_2, f_3 and c each satisfy a tribonacci recurrence (the c-exponent with an extra $2x - 6$ term, handled by adding x and 1 coordinates). By Fermat, exponents are reduced **mod p − 1**, not mod p. This is a classic trap.

2.3 Larger orders

When k is in the hundreds or thousands, $k^3 \log n$ is too slow. The recurrence term a_n equals a linear functional applied to $x^n \bmod P(x)$ where P is the characteristic polynomial (Kitamasa's method), giving $O(k^2 \log n)$ naively or $O(k \log k \log n)$ with NTT-based polynomial modulo. If the recurrence is unknown but you can generate 2k terms, Berlekamp–Massey finds it in $O(k^2)$. Both are covered in the World Finals track.

Many queries, same matrix. Precompute M^{2^j} once ($O(k^3 \log n)$); each query multiplies a *vector* by $O(\log n)$ matrices — $O(k^2 \log n)$ per query instead of $O(k^3 \log n)$.

3. FFT and NTT

3.1 Why it works

The product $C = A \cdot B$ has coefficients $c_k = \Sigma_{i+j=k} a_i b_j$ — a convolution. A polynomial of degree < N is determined by its values at N points, and the values of C are products of the values of A and B. Choose the points to be the N-th roots of unity $\omega^0, \ldots, \omega^{N-1}$ with N a

power of two ≥ deg C + 1. Then:

- **Evaluation** splits $A(x) = A_0(x^2) + x\,A_1(x^2)$ (even/odd coefficients). Squares of N-th roots are the N/2-th roots, each appearing twice, so $T(N) = 2T(N/2) + O(N) = O(N \log N)$. The combine step is the butterfly: with $u = A_0(\omega^{2j})$, $v = \omega^j A_1(\omega^{2j})$, we get $A(\omega^j) = u + v$ and $A(\omega^{j+N/2}) = u - v$ because $\omega^{N/2} = -1$.
- **Interpolation** is the same transform with ω^{-1}, divided by N. Proof: $\Sigma_{k=0}^{N-1} \omega^{jk}$ equals N if $j \equiv 0 \pmod N$ and 0 otherwise (geometric series), so the DFT matrix times its conjugate is N·I.

The iterative implementation first permutes the input into bit-reversed order (the leaf order of the recursion) and then runs layers $len = 2, 4, \ldots, N$.

3.2 NTT

Over the field Z/p, a primitive N-th root of unity exists iff N divides p − 1. For $p = 998244353 = 119 \cdot 2^{23} + 1$ with primitive root 3, $3^{(p-1)/N}$ is a primitive N-th root for every power of two $N \le 2^{23}$. The same butterfly code works with modular arithmetic and is exact. Other common NTT primes: $469762049 = 7 \cdot 2^{26}+1$ and $167772161 = 5 \cdot 2^{25}+1$, both with root 3.

For an arbitrary modulus such as 10^9+7, either convolve modulo three NTT primes and recombine with CRT (the true coefficient is $< n \cdot (10^9)^2$ < product of the three primes), or use complex FFT on 15-bit halves of each coefficient.

3.3 Complex FFT precision

With `double`, coefficients of the result are exact after rounding if they stay below roughly 10^{14}–10^{15} and $N \le \sim 10^6$, provided twiddle factors are computed accurately (`polar(1.0, ang*j)` or a precomputed table; repeated multiplication `w *= wlen` accumulates error over long blocks). Always round with `llround`.

3.4 Spotting convolutions

- "For every s, count pairs (i, j) with $x_i + y_j = s$" — convolve the count arrays. Example: CF 993E *Nikita and Order Statistics* convolves counts of prefix positions with each number of "small" elements, against the reversed array.
- "Match pattern against every shift of text" — reverse the pattern so that the sum over aligned positions becomes a convolution index. Per-letter indicator convolutions count matches; wildcard matching uses $\Sigma\ p_j\ t_{i+j}\ (p_j - t_{i+j})^2$, which expands into three convolutions (wildcards encoded as 0).
- CF 528D *Fuzzy Search*: for each of the 4 letters, mark positions in T where that letter occurs within distance k (a sweep), convolve with the reversed indicator of that letter in P, and a shift is good iff the four match counts sum to |P|.

- Products of many small polynomials (generating functions for "choose items from groups") — multiply them in a divide-and-conquer tree, total O(N log² N). CF 1251F *Red-White Fence* is a good exercise.

4. Gaussian elimination

To solve `A x = b` with A of size n×m: for each column, pick a pivot row with a nonzero entry, swap it up, normalize, and eliminate that column from **all** other rows (Gauss–Jordan). Columns without a pivot are free variables. After the sweep, a row that is all zeros on the left and nonzero on the right means the system is inconsistent; otherwise one solution is obtained by setting free variables to 0, and the solution set has size `|F|^{m − rank}` over a finite field F. Complexity O(n·m·min(n, m)).

Over the reals choose the pivot with the largest absolute value (partial pivoting) and treat |x| < ε as zero; without this, round-off destroys the answer. Typical use: expected values on a Markov chain with cycles (CF 24D *Broken robot* is a tridiagonal special case solved row by row in O(m) per row).

Mod p any nonzero pivot works; normalize with the modular inverse. Typical uses: counting solutions (p^{free}), polynomial interpolation from point values (CF 1155E *Guess the Root* asks 11 queries and solves the 11×11 system mod 10^6+3), and puzzles like CF 832E *Vasya and Shifts* (mod 5).

Over GF(2) addition is xor, so rows are `bitset<N>` and elimination costs $O(n^2 m / 64)$. Lights-out style puzzles and parity constraints live here. The determinant over any field is the product of the pivots times (−1)^{#swaps}.

5. The XOR basis

Gaussian elimination over GF(2) on numbers viewed as bit vectors, organized by highest set bit, gives the **xor basis**. We keep `b[i]` = a vector whose highest set bit is i (or 0). To insert x, scan bits from high to low; whenever bit i of x is set, either store x in the empty slot `b[i]` or replace x by `x ⊕ b[i]` (clearing bit i). If x becomes 0 it was already in the span.

Invariants. Each step is an invertible row operation, so span{b} always equals the span of all inserted numbers, and the nonzero `b[i]` are linearly independent (distinct leading bits). Therefore:

- The number of distinct subset-xor values is 2^r, where r is the rank, and each value is produced by exactly 2^{n−r} of the 2^n subsets (the kernel of the map "subset ↦ xor" has dimension n − r). This counts subsets with xor x (CF 959F).
- **Max xor**: go from high bit to low, `res = max(res, res ⊕ b[i])`. Greedy is correct because b[i] is the only basis vector that can change bit i once higher bits are fixed. Starting from

`res = s` gives the maximum of `s ⊕ (subset xor)` — exactly what xor-path problems need.
- **Membership**: reduce x; it is in the span iff it becomes 0.
- **k-th smallest**: first reduce the basis to reduced row echelon form (each leading bit appears in only one vector). Then the span, sorted, is indexed by the binary digits of k: the i-th smallest leading-bit vector is selected iff bit i of k is set.

Patterns.

- *Xor of a path in a graph* (CF 845G *Shortest Path Problem?*): fix a DFS tree with root-to-vertex xors `d[v]`. Every walk from u to v has xor `d[u] ⊕ d[v] ⊕ (xor of some set of cycles)`, and the cycle space is spanned by fundamental cycles `d[a] ⊕ d[b] ⊕ w` for non-tree edges. Insert those into a basis and minimize `d[1] ⊕ d[n]` (a "min" variant of the greedy).
- *Range queries* (CF 1100F *Ivan and Burgers*): process r increasing, keeping for each bit the basis vector with the **largest position**; when inserting a vector with a newer position, swap it with the stored older one and continue inserting the older one. A query (l, r) uses only basis vectors with position ≥ l. Proof sketch: this basis is, for every l simultaneously, a basis of the suffix a[l..r]. O((n+q) log C).
- *Rank as the answer* (CF 1101G): splitting into segments with no zero-xor set of segments is possible with as many segments as the rank of the prefix xors, provided the total xor is nonzero.
- *Square-free products* (CF 895C *Square Subsets*): map each number to the parity vector of its prime exponents; the count of subsets with product a perfect square is 2^{n − r} − 1.

6. Reference implementations

All five programs compile with `g++ -std=c++17 -O2 -Wall` and were tested against brute force on hundreds of random inputs (recurrences against direct iteration, NTT/FFT against O(nm) convolution — FFT also cross-checked against NTT for n = 2·10⁵ — Gaussian elimination against enumeration of all solutions mod small primes, xor basis against explicit span enumeration).

6.1 Linear recurrence via matrix power

Input: `k n`, then coefficients `c_1..c_k`, then `a_0..a_{k−1}`. Output `a_n mod 10⁹+7`.

```
#include <bits/stdc++.h>
using namespace std;
const long long MOD = 1'000'000'007;
using Mat = vector<vector<long long>>;

Mat mul(const Mat& A, const Mat& B) {
    int n = A.size(), m = B[0].size(), k = B.size();
    Mat C(n, vector<long long>(m, 0));
    for (int i = 0; i < n; i++)
        for (int t = 0; t < k; t++) {
```

```
            if (A[i][t] == 0) continue;
            for (int j = 0; j < m; j++) C[i][j] = (C[i][j] + A[i][t] * B[t][j]) % MOD;
        }
    return C;
}

Mat power(Mat A, long long e) {
    int n = A.size();
    Mat R(n, vector<long long>(n, 0));
    for (int i = 0; i < n; i++) R[i][i] = 1;
    while (e > 0) {
        if (e & 1) R = mul(R, A);
        A = mul(A, A);
        e >>= 1;
    }
    return R;
}

// a[i] = c[0]*a[i-1] + c[1]*a[i-2] + ... + c[k-1]*a[i-k]; given a[0..k-1], return a[n].
long long nthTerm(const vector<long long>& c, const vector<long long>& init, long long n)
{
    int k = c.size();
    if (n < k) return init[n] % MOD;
    Mat T(k, vector<long long>(k, 0));        // companion matrix
    for (int j = 0; j < k; j++) T[0][j] = c[j] % MOD;
    for (int i = 1; i < k; i++) T[i][i - 1] = 1;
    Mat P = power(T, n - k + 1);
    // state vector (a[k-1], a[k-2], ..., a[0])^T
    long long res = 0;
    for (int j = 0; j < k; j++) res = (res + P[0][j] * (init[k - 1 - j] % MOD)) % MOD;
    return res;
}

int main() {
    int k; long long n;
    cin >> k >> n;
    vector<long long> c(k), init(k);
    for (auto& x : c) cin >> x;
    for (auto& x : init) cin >> x;
    cout << nthTerm(c, init, n) << "\n";
}
```

6.2 NTT convolution mod 998244353

Input: `n m`, then the n coefficients of A and the m coefficients of B. Output the n+m−1 coefficients of A·B mod 998244353. Runs in about 0.3 s for $n = m = 5 \cdot 10^5$.

```
#include <bits/stdc++.h>
using namespace std;
const int MOD = 998244353, G = 3;   // 998244353 = 119 * 2^23 + 1, primitive root 3

long long power(long long b, long long e) {
```

```cpp
    long long r = 1; b %= MOD;
    for (; e > 0; e >>= 1, b = b * b % MOD) if (e & 1) r = r * b % MOD;
    return r;
}

void ntt(vector<int>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {          // bit-reversal permutation
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        long long w = power(G, (MOD - 1) / len);
        if (invert) w = power(w, MOD - 2);
        for (int i = 0; i < n; i += len) {
            long long wn = 1;
            for (int j = 0; j < len / 2; j++) {
                int u = a[i + j], v = a[i + j + len / 2] * wn % MOD;
                a[i + j] = u + v < MOD ? u + v : u + v - MOD;
                a[i + j + len / 2] = u - v >= 0 ? u - v : u - v + MOD;
                wn = wn * w % MOD;
            }
        }
    }
    if (invert) {
        long long inv = power(n, MOD - 2);
        for (int& x : a) x = x * inv % MOD;
    }
}

vector<int> multiply(vector<int> a, vector<int> b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <<= 1;
    a.resize(n); b.resize(n);
    ntt(a, false); ntt(b, false);
    for (int i = 0; i < n; i++) a[i] = 1LL * a[i] * b[i] % MOD;
    ntt(a, true);
    a.resize(need);
    return a;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<int> a(n), b(m);
    for (auto& x : a) cin >> x;
    for (auto& x : b) cin >> x;
    vector<int> c = multiply(a, b);
    for (size_t i = 0; i < c.size(); i++) cout << c[i] << " \n"[i + 1 == c.size()];
}
```

6.3 Complex FFT for exact integer products

Same input format; coefficients may be negative. Valid while result coefficients stay below about 10^{14} in absolute value.

```cpp
#include <bits/stdc++.h>
using namespace std;
using cd = complex<double>;
const double PI = acos(-1.0);

void fft(vector<cd>& a, bool invert) {
    int n = a.size();
    for (int i = 1, j = 0; i < n; i++) {
        int bit = n >> 1;
        for (; j & bit; bit >>= 1) j ^= bit;
        j ^= bit;
        if (i < j) swap(a[i], a[j]);
    }
    for (int len = 2; len <= n; len <<= 1) {
        double ang = 2 * PI / len * (invert ? -1 : 1);
        for (int i = 0; i < n; i += len)
            for (int j = 0; j < len / 2; j++) {
                cd w = polar(1.0, ang * j);              // exact twiddles: better
precision
                cd u = a[i + j], v = a[i + j + len / 2] * w;
                a[i + j] = u + v;
                a[i + j + len / 2] = u - v;
            }
    }
    if (invert) for (cd& x : a) x /= n;
}

// Exact product of integer polynomials while every coefficient of the result
// stays well below ~1e14 in absolute value.
vector<long long> multiply(const vector<long long>& a, const vector<long long>& b) {
    if (a.empty() || b.empty()) return {};
    int need = a.size() + b.size() - 1, n = 1;
    while (n < need) n <<= 1;
    vector<cd> fa(a.begin(), a.end()), fb(b.begin(), b.end());
    fa.resize(n); fb.resize(n);
    fft(fa, false); fft(fb, false);
    for (int i = 0; i < n; i++) fa[i] *= fb[i];
    fft(fa, true);
    vector<long long> res(need);
    for (int i = 0; i < need; i++) res[i] = llround(fa[i].real());
    return res;
}

int main() {
    ios::sync_with_stdio(false); cin.tie(nullptr);
    int n, m;
    cin >> n >> m;
    vector<long long> a(n), b(m);
```

```
    for (auto& x : a) cin >> x;
    for (auto& x : b) cin >> x;
    vector<long long> c = multiply(a, b);
    for (size_t i = 0; i < c.size(); i++) cout << c[i] << " \n"[i + 1 == c.size()];
}
```

6.4 Gauss–Jordan elimination mod a prime

Input: `n m p`, then n rows of m+1 integers (augmented matrix). Output `inconsistent`, or the rank, number of free variables and one solution.

```
#include <bits/stdc++.h>
using namespace std;

long long power(long long b, long long e, long long m) {
    long long r = 1; b %= m;
    for (; e > 0; e >>= 1, b = b * b % m) if (e & 1) r = r * b % m;
    return r;
}

// Solve A x = b over GF(p), p prime. a is n x (m+1) augmented matrix.
// Returns -1 if inconsistent, otherwise the rank; x receives one solution
// (free variables set to 0). Number of solutions = p^(m - rank).
int gaussMod(vector<vector<long long>> a, long long p, vector<long long>& x) {
    int n = a.size(), m = a[0].size() - 1;
    vector<int> where(m, -1);
    int row = 0;
    for (int col = 0; col < m && row < n; col++) {
        int sel = -1;
        for (int i = row; i < n; i++) if (a[i][col] % p != 0) { sel = i; break; }
        if (sel == -1) continue;                    // free column
        swap(a[sel], a[row]);
        where[col] = row;
        long long inv = power(a[row][col], p - 2, p);
        for (int j = col; j <= m; j++) a[row][j] = a[row][j] * inv % p;
        for (int i = 0; i < n; i++) {
            if (i == row || a[i][col] == 0) continue;
            long long f = a[i][col];
            for (int j = col; j <= m; j++) a[i][j] = ((a[i][j] - f * a[row][j]) % p + p)
% p;
        }
        row++;
    }
    for (int i = row; i < n; i++) if (a[i][m] % p != 0) return -1;   // 0 = nonzero
    x.assign(m, 0);
    for (int col = 0; col < m; col++) if (where[col] != -1) x[col] = a[where[col]][m];
    return row;
}

int main() {
    int n, m; long long p;
    cin >> n >> m >> p;
```

```
    vector<vector<long long>> a(n, vector<long long>(m + 1));
    for (auto& r : a) for (auto& v : r) { cin >> v; v = ((v % p) + p) % p; }
    vector<long long> x;
    int r = gaussMod(a, p, x);
    if (r < 0) { cout << "inconsistent\n"; return 0; }
    cout << "rank " << r << " free " << m - r << "\n";
    for (int i = 0; i < m; i++) cout << x[i] << " \n"[i + 1 == m];
}
```

6.5 XOR basis

Input: `n q`, n numbers ($< 2^{60}$), then q queries `x k`. Output rank and maximum subset xor, then per query whether x is a subset xor and the k-th smallest (0-indexed, empty subset included) subset xor.

```
#include <bits/stdc++.h>
using namespace std;

// Linear basis over GF(2) for 60-bit numbers (Gaussian elimination on bits).
struct XorBasis {
    static const int B = 60;
    long long b[B] = {};         // b[i] has highest set bit i, or is 0
    int rank = 0;
    bool insert(long long x) {
        for (int i = B - 1; i >= 0; i--) {
            if (!(x >> i & 1)) continue;
            if (!b[i]) { b[i] = x; rank++; return true; }
            x ^= b[i];
        }
        return false;            // x was already in the span
    }
    bool contains(long long x) const {
        for (int i = B - 1; i >= 0; i--)
            if (x >> i & 1) { if (!b[i]) return false; x ^= b[i]; }
        return true;
    }
    long long maxXor(long long start = 0) const {
        long long r = start;
        for (int i = B - 1; i >= 0; i--) r = max(r, r ^ b[i]);
        return r;
    }
    // k-th smallest (0-indexed) element of the span, k < 2^rank
    long long kth(long long k) const {
        vector<long long> red;                    // reduced row echelon form
        long long c[B];
        copy(b, b + B, c);
        for (int i = 0; i < B; i++) if (c[i])
            for (int j = i + 1; j < B; j++) if (c[j] >> i & 1) c[j] ^= c[i];
        for (int i = 0; i < B; i++) if (c[i]) red.push_back(c[i]);
        long long r = 0;
        for (int i = 0; i < (int)red.size(); i++) if (k >> i & 1) r ^= red[i];
        return r;
```

```
    }
};

int main() {
    int n, q;
    cin >> n >> q;
    XorBasis xb;
    for (int i = 0; i < n; i++) { long long v; cin >> v; xb.insert(v); }
    cout << xb.rank << " " << xb.maxXor() << "\n";
    while (q--) {
        long long x, k;
        cin >> x >> k;                // membership query and k-th smallest query
        cout << xb.contains(x) << " " << xb.kth(k % (1LL << xb.rank)) << "\n";
    }
}
```

7. Common mistakes

- **Overflow in matrix products.** Each product term is $< \text{MOD}^2 \approx 10^{18}$; reduce after every multiply-add. Accumulating several terms before reducing overflows `long long`.
- **Wrong modulus for exponents.** `a^e mod p` with e itself huge: reduce e mod p − 1 (if gcd(a, p) = 1). Take care when a ≡ 0 mod p.
- **Semiring identity.** For (min, +) powers the identity has 0 on the diagonal and ∞ elsewhere; using 1 on the diagonal silently adds weight.
- **Off-by-one in the companion power.** Write down which vector the power maps to which; test n < k separately.
- **NTT length.** The transform size must be a power of two ≥ n + m − 1 (not ≥ max(n, m)) and must divide p − 1. Input values must be reduced into [0, p).
- **Wrong NTT prime for the problem.** If the problem's modulus is 10^9+7, a single NTT is wrong; use three-prime CRT or split FFT.
- **FFT precision.** Coefficients $\sim 10^{18}$ cannot be exact in doubles. Estimate `max|c_k| ≤ N · max|a| · max|b|` before choosing FFT.
- **No pivoting over the reals**, or comparing to exactly 0.0.
- **Xor basis with `1 << i` for i ≥ 31.** Use `1LL << i` or shift the value (`x >> i & 1`).
- **Forgetting the empty subset.** The span always contains 0 (the empty subset). If the statement requires a *non-empty* subset, 0 is achievable only when some insertion failed (rank < n); adjust k-th / min queries accordingly.

8. Practice problems

Matrix exponentiation:

- CSES 1722 — Fibonacci Numbers — https://cses.fi/problemset/task/1722
- CSES 1096 — Throwing Dice — https://cses.fi/problemset/task/1096
- CSES 1723 — Graph Paths I — https://cses.fi/problemset/task/1723

- CSES 1724 — Graph Paths II — https://cses.fi/problemset/task/1724
- Codeforces 222E — Decoding Genome — https://codeforces.com/problemset/problem/222/E
- Codeforces 1182E — Product Oriented Recurrence — https://codeforces.com/problemset/problem/1182/E
- Codeforces 576D — Flights for Regular Customers (stretch) — https://codeforces.com/problemset/problem/576/D

FFT / NTT:

- AtCoder ATC001 C — 高速フーリエ変換 — https://atcoder.jp/contests/atc001/tasks/fft_c
- Library Checker — Convolution — https://judge.yosupo.jp/problem/convolution_mod
- Codeforces 528D — Fuzzy Search — https://codeforces.com/problemset/problem/528/D
- Codeforces 993E — Nikita and Order Statistics — https://codeforces.com/problemset/problem/993/E
- Codeforces 1251F — Red-White Fence — https://codeforces.com/problemset/problem/1251/F

Gaussian elimination and xor basis:

- Codeforces 24D — Broken robot — https://codeforces.com/problemset/problem/24/D
- Codeforces 832E — Vasya and Shifts — https://codeforces.com/problemset/problem/832/E
- Codeforces 1155E — Guess the Root — https://codeforces.com/problemset/problem/1155/E
- SPOJ XMAX — XOR Maximization — https://www.spoj.com/problems/XMAX/
- Codeforces 845G — Shortest Path Problem? — https://codeforces.com/problemset/problem/845/G
- Codeforces 959F — Mahmoud and Ehab and yet another xor task — https://codeforces.com/problemset/problem/959/F
- Codeforces 1101G — (Zero XOR Subset)-less — https://codeforces.com/problemset/problem/1101/G
- Codeforces 895C — Square Subsets — https://codeforces.com/problemset/problem/895/C
- Codeforces 1100F — Ivan and Burgers — https://codeforces.com/problemset/problem/1100/F

9. Further reading

- cp-algorithms: "Binary exponentiation" (matrix section), "Fast Fourier transform", "Gauss method for solving system of linear equations" — https://cp-algorithms.com/
- T. H. Cormen et al., *Introduction to Algorithms*, chapter "Polynomials and the FFT".
- The official editorial of CF 1100F for the position-preferring ("prefix") linear basis, and of CF 845G for the cycle-space argument.
- KACTL (KTH team notebook) — `FastFourierTransform.h`, `NumberTheoreticTransform.h`, `SolveLinear.h`, `MatrixInverse.h` for compact, battle-tested versions: https://github.com/kth-competitive-programming/kactl

- World Finals track, Lecture 2 (polynomial inverse/log/exp, Berlekamp–Massey, Lagrange interpolation) continues directly from this lecture.